Posted on 29 Apr 2025 — The copyright holder is the author/funder. All rights reserved. No reuse without permission. — https://doi.org/10.22541/au.174593665.52735213/v1 — This is a preprint and has not been peer-reviewed. Data may be preliminary.

Optimizing Server and Memory Utilization in Cloud Computing through Virtualization and Caching

V. V. Jaya Rama Krishnaiah[1] and B. Srinivasa Rao[2]

[1]Koneru Lakshmaiah Education Foundation
[2]Teegala Krishna Reddy Engineering College

April 29, 2025

Abstract

Modern cloud computing empowers users to dynamically adjust their resource consumption, while data centers utilize virtualization to manage resource allocation. This paper introduces a system that leverages virtualization to allocate memory to resources based on end-user requirements, employing cache memory for optimization. Our solution tackles the under-utilization of servers, a common challenge in contemporary cloud computing environments. Although various techniques have been developed to address this issue, achieving maximum server utilization remains difficult due to the increasing number of end-users.

Introduction

Cloud computing, characterized by the concept of "computer-as-a-service," is a prominent feature of today's internet-based applications. It offers on-demand services, including computing resources like CPUs, software, data, and devices, to end-users. Cloud computing enables diverse business environments to efficiently scale their IT needs, providing a versatile, scalable, efficient, and cost-effective solution. Major companies such as Amazon, Microsoft, IBM, and Google are heavily investing in large data centers. These data centers lease physical resources, including CPU, disk space, memory, and network capacity, based on the immediate requirements of client companies. The dynamic nature of resource demands in these organizations increases the complexity of resource management in data centers.

The rapid growth of internet-based applications has led to a surge in users, intensifying competition among data centers. Storing user data is a critical challenge for organizations, prompting them to rely on cloud providers who utilize advanced technologies to manage large volumes of client data. Virtualization provides a solution for data centers to effectively manage these challenges, with hypervisors enabling multiple virtual machines on physical machines.

Cloud providers address server over-provisioning in different ways. Some allocate generous physical resources to each virtual machine to handle peak load demands, while others use fewer resources. Resource oversubscription, wherein multiple VMs share a single physical server, may result in performance degradation once aggregate VM demands exceed host capacity. Mitigating such overloads is critical for competitive cloud service provisioning.

This paper focuses on memory overload in physical machines. Although memory cannot be oversubscribed like CPU or I/O, its overloading significantly impacts application performance.

Reasons for Overloading

This section identifies the various reasons for overloading in different data centers and examines their behavior by analyzing data center logs from large organizations. In a cloud computing environment, overloading can be caused by both cloud users and cloud providers. Virtual machines are overloaded when the allocated physical

memory is significantly insufficient to process the applications requested by the cloud user. Sometimes, cloud providers may face challenges when servers are underutilized because servers are overloaded with over-provisioning of virtual machines on a single physical machine. These are two common memory issues in cloud computing-enabled systems.

First, it is important to define memory allocation, memory allocation represents a fundamental aspect of cloud infrastructure, wherein virtual memory is provisioned by the provider to support client workloads. Despite increasing user access, server underutilization remains a concern. Virtual machines are generally adaptable in memory usage, though application-specific demands can exceed predefined limits, thereby impacting overall performance.

This is related to memory ballooning. If virtual machines are overloaded, their physical machine servers may be underutilized. Therefore, cloud providers are responsible for ensuring that cloud users have sufficient memory as requested. Managing virtual machines and server underutilization creates overhead for cloud providers. Cloud users are often unaware of overloading and over-subscription.

Data centers commonly use two indices to measure server overloading: page scan rate and paging rate. Paging rate is a basic index that deals with the operating system's success in obtaining free pages. The operating system logs any unused free pages, and the paging rate deals with these logs. The page scan rate provides information about memory utilization. An increased page scan rate indicates significant server underutilization.

Increase of Overloading due to Oversubscription: To determine the effects of overloading with oversubscription, it is necessary to understand the client load in relation to the servers. In some computer systems, the client load involves a user entering the system as another exits, such as with ATM machines. However, real-time systems like flight or railway booking systems do not have a static number of users. User surges can stall the entire server due to many users simultaneously requesting the same resources. Therefore, end-user load in real-time systems varies. Poisson's ratio is used to measure the proportional ratio of arrivals and exits. Real-time systems are prone to unpredictable overloads.

Migration of Virtual Machines: One technique to address overloading is migrating virtual machines on the same physical machines. Adequate memory for a virtual machine on a mapped physical machine prevents overloading. This means that allocating more memory to cloud users as requested can resolve overloading. However, virtual machine migration is resource-intensive. Key migration strategies include resource utilization, overload prediction, and placement strategies.

Migration strategies are popular because they ensure that all application components have sufficient memory to complete execution without performance degradation. Previously, some internal flash memories were used as I/O buffers for frequently used data. However, this implementation of flash memories was not successful due to insufficient cache size. This work differs from managing I/O or local and remote disks by using coherent caching.

Coherent caching is a subset application of network memory, which performs faster than disk, especially on different data center networks.

Coherent Caching: Coherent caching uses the most recently used and highly frequently used memory pages from network memory. In this technique, a page repository across the network stores memory addresses (pages) to prevent further memory allocation to already served requests. The amount of memory pages read or written from the network depends on the duration of the overload. Figure 1 illustrates the hardware node structure.

Hardware Node

Posted on 29 Apr 2025 — The copyright holder is the author/funder. All rights reserved. No reuse without permission. — https://doi.org/10.22541/au.174593665.52735213/v1 — This is a preprint and has not been peer-reviewed. Data may be preliminary.

Posted on 29 Apr 2025 — The copyright holder is the author/funder. All rights reserved. No reuse without permission. — https://doi.org/10.22541/au.174593665.52735213/v1 — This is a preprint and has not been peer-reviewed. Data may be preliminary.

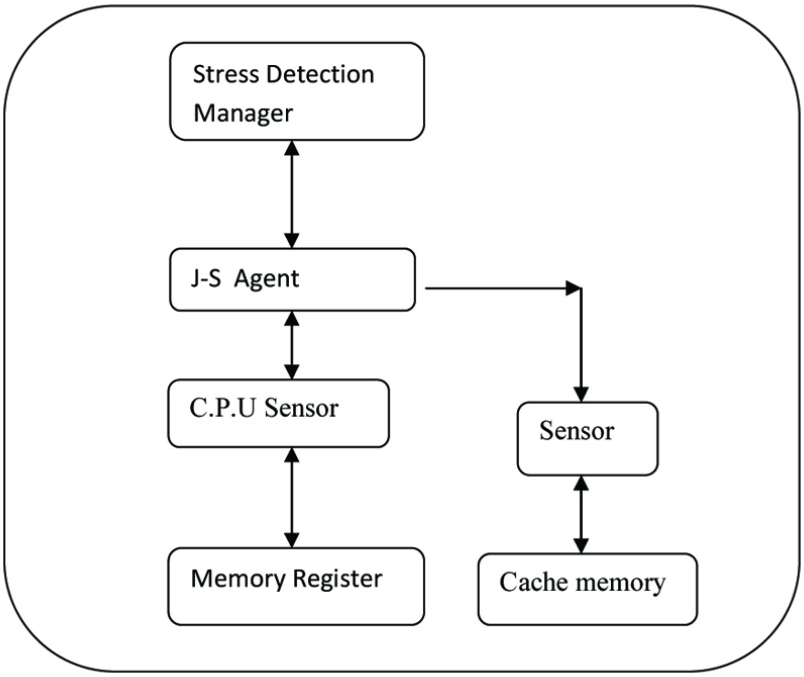

Fig 1: Hardware Node Structure

Related Work

This section examines mitigation strategies for addressing the overloading problem. An architecture that can handle memory overloading is proposed, introducing the concept of coherent caching.

Proposed Architecture

How Memory is Allocated: In the proposed architecture, a set of sensors on a hardware node is managed by a Stress Detection Manager. The detection management agent provides a standard interface to the sensors. The schedule manager in the system manager receives the sensor data and analyzes it to determine appropriate memory allocation actions.

Dynamically allocating memory to virtual machines mapped to physical machines in a cloud computing environment is challenging. The allocation should enable the system to self-adjust to the workload without violating the Service Level Agreement (SLA). The mapping of virtual machines to physical machines is as follows. The physical servers are assumed to be homogeneous, while the physical machines' resources, such as CPU, disk, and memory, are heterogeneous. Resources can be managed by scaling or migrating as appropriate. Scaling is a widely used approach. To simplify the problem formulation without changing the existing solution, it is noted that in overloading scenarios, the resources used by the virtual machines (CPU, memory, disk, and I/O) can lead to SLA violations.

Problem Formulation

Each virtual machine has a d-dimensional vector, and the resource utilization of each d-dimensional vector can be represented as VMi (i = 1, 2, 3, . . . , n). Each physical machine is represented as PMj (j = 1, 2, 3, . . . , n), and the utilization vector can be represented as ui with respect to time t. The combinational vector of physical machine and utilization is Lk(t), where k is a variable, t is time, and L represents the combination of

Posted on 29 Apr 2025 — The copyright holder is the author/funder. All rights reserved. No reuse without permission. — https://doi.org/10.22541/au.174593665.52735213/v1 — This is a preprint and has not been peer-reviewed. Data may be preliminary.

physical machine and server utilization at a point in time. In this d-dimensional vector space, cache physical machines have a capacity vector Cj. There are assumed to be n virtual machines on a physical machine.

Virtual machines are initially allocated to physical machines at t=0 with a predefined allocation. As more virtual machines are allocated, the overloading of virtual machines on the physical machines will exceed the threshold limit of predictive allocation. The algorithm runs at discrete time intervals t0, t1, . . . , tk, up to k undefined intervals, until overloading occurs. SLA is mapped to dynamic resource allocation. If this dynamic resource allocation causes overloading, the coherent caching procedure is triggered. Otherwise, virtualization will function normally. Several authors have proposed reallocation procedures to address overloading. However, reallocation can lead to SLA violations and is resource-intensive and costly because it requires restarting the allocation of different virtual machines to be mapped on physical machines.

The input for the algorithm in a real-world scenario is:

Lk(t) = {u1(t), u2(t), . . . , un(t)}

Here, Lk(t) is the function of physical machine utilization with respect to time t.

Js(t) = {u1(t) + PM1, u2(t) + PM2, . . . , un(t) + PMn}

Here, Js(t) is the combinational vector of virtual machine and physical machine utilization. The combinational vector Js(t) may not be accurate for all system resources like CPU and disk. A virtual machine's resource utilization at time t can be denoted by:

Uj(t) = f{uj1(t) + uj2(t) + . . . + ujk(t)} (1)

Where ujk = nth utilization of the kth virtual machine.

When virtual machines are mapped with different physical machines, the set of physical machines can be denoted by a set A:

Physical machine = A{PMj(j) = 1, 2, . . . , k} (2)

The ith resource of the physical machine can be defined as:

PMi = {PM1 + PM2 + . . . + PMj} (3)

By deriving a combinational vector of the virtual machine allocation and the ith resource of the physical machine from equations (1), (2), and (3), the load of the jth virtual machine, which has been mapped to the kth physical machine, is:

PMk = [PMk(i) + uj(t)]

Where:

i = ith resource of kth Physical Machine

t = time of the jth virtual machine.

Assuming Cj is the fixed capacity of the physical machine, overloading occurs when the combinational vector Jc exceeds Cj.

By finding the threshold limit of the cache physical machine in the data centers, a probability is derived:

P = {Jc(t) > Cj} (4)

Where t = time

If the probability function P continues to increase, overloading is triggered, leading to increased oversubscription of virtual machines with respect to the ith resource of the kth physical machine, which can degrade performance.

Therefore, data centers should focus on the probability function:

Posted on 29 Apr 2025 — The copyright holder is the author/funder. All rights reserved. No reuse without permission. — https://doi.org/10.22541/au.174593665.52735213/v1 — This is a preprint and has not been peer-reviewed. Data may be preliminary.

$P = \{Cj > Jc(t) / \varepsilon oi\}$ (5)

Where εoi = overloading at the ith instance.

The capacity Cj should always be greater than the combinational vector Jc(t). The algorithm checks equations (4) and (5) to determine when to proceed with further resource allocation, migrate virtual machines, or initiate coherent caching.

Mc is the virtual machine migration cost of the kth virtual machine. The goal is to keep this cost metric low. The cost of migrating the kth virtual machine, vmj, is [Mc, uj(t)].

The objective is to allocate memory to the virtual machine without violating the SLA across different time intervals t0, t1, t2, . . . , tn. IBM Director is used to monitor this. The algorithm aims to direct the Joint Solution (J-S) agent to take appropriate action: when to migrate the virtual machine and when to initiate coherent caching. The algorithm monitors:

1. Which virtual machine should be removed from the physical machine.
2. Where to migrate the removed virtual machine.
3. When to initiate coherent caching.
4. How to map the new physical machine to the migrated virtual machine.

SLA violations can occur at any time during application processing. The algorithm initiates the optimal steps for the given time. If the CPU utilization (O Pi(t) < 100) is the ith machine, and given two vectors, Pt: and , where the first represents virtual machine memory capacity and the second represents available resources at the ith instance, the residual vector (capacity vector Cj of the physical machine) is high [10, 100, 90]. The capacity Cj of the ith physical machine should accommodate the virtual machine.

Experiments and Results

Server overloading in well-organized data centers is unexpected, rare, and uncorrelated. An experiment was conducted to evaluate memory overloading in data centers with 80 randomly selected servers over 12 hours [3 + 6 + 3]. The log included parameters such as CPU, memory, and I/O. Cloud data centers host various applications, including websites, customer relationship management systems, financial applications, and educational platforms. Hosting these diverse applications requires constant observation by cloud providers to maintain the servers.

The experiment found that 20 servers were overloaded, compared to 15 servers experiencing relatively less overloading out of the 80 servers. However, the 15 servers were overloaded simultaneously, with an average overload duration of 2.17 hours over the 12-hour period. This indicates that sharing server resources on physical machines leads to simultaneous overloading.

Posted on 29 Apr 2025 — The copyright holder is the author/funder. All rights reserved. No reuse without permission. — https://doi.org/10.22541/au.174593665.52735213/v1 — This is a preprint and has not been peer-reviewed. Data may be preliminary.

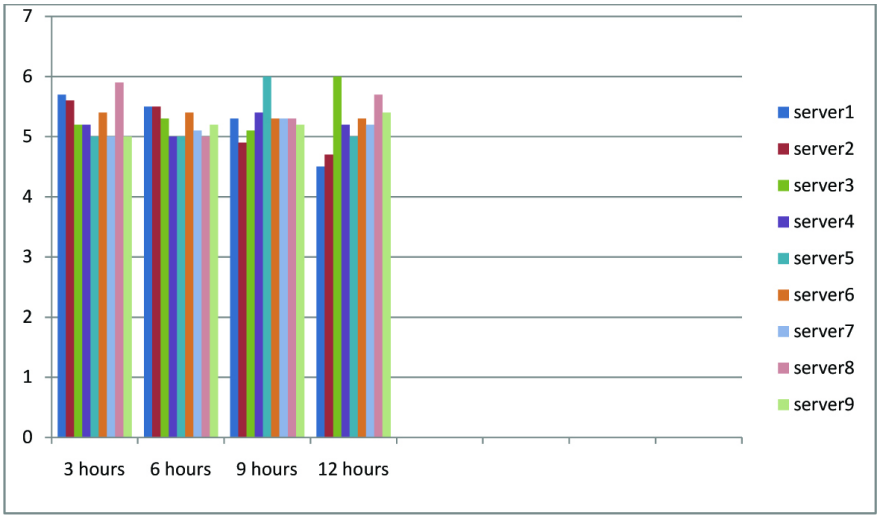

Fig 2: Functional behaviour of servers with different time slots

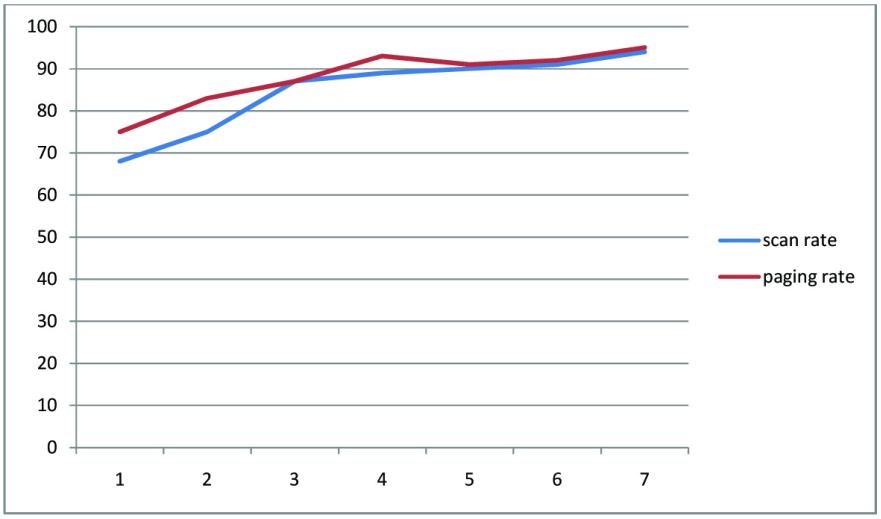

Fig 3: Overloading of Scan rate and paging rate comparison

Figure 2 illustrates server behavior over time. Figure 3 compares overloading of scan rate and paging rate.

According to the document, 68% of the servers were overloaded in the first time interval, 75% (68% + 7%) in the next time slot, and 87% (68% + 7% + 12%) in the third interval, with respect to [10, 20, 30] minutes.

J-S Agent: The J-S agent was introduced to differentiate between virtual machine migration and coherent caching.

In the proposed architecture, each physical machine in the hardware node runs on a hypervisor and hosts several virtual machines. The hardware node also includes the J-S (Joint Solution) agent, which controls and monitors memory overload using page rate and page scan rate indices. Predefined thresholds determine whether the J-S agent initiates coherent caching or virtual machine migration. Sustained overloads, which severely affect physical machines, are addressed using the J-S agent's Joint Control (J-C) strategy. J-C aims to address overload without migrating (oversubscribing) virtual machines. The joint controller tracks

available resources and the addresses of unallocated or completed memory pages, maintaining a repository of free memory pages. Transient overload durations generally do not exceed the threshold limit, allowing coherent caching to resolve the overload.

If the joint controller finds available memory pages and free resources, it sends this information to the joint solution agent. The J-S agent decides on memory availability and redirects the virtual machines to use the memory space. The J-S agent also calculates if the available memory and resources are sufficient to process the current request. If so, no migration occurs. Otherwise, the J-S agent initiates coherent caching. If this fails, the overload is considered sustained, and migration is initiated.

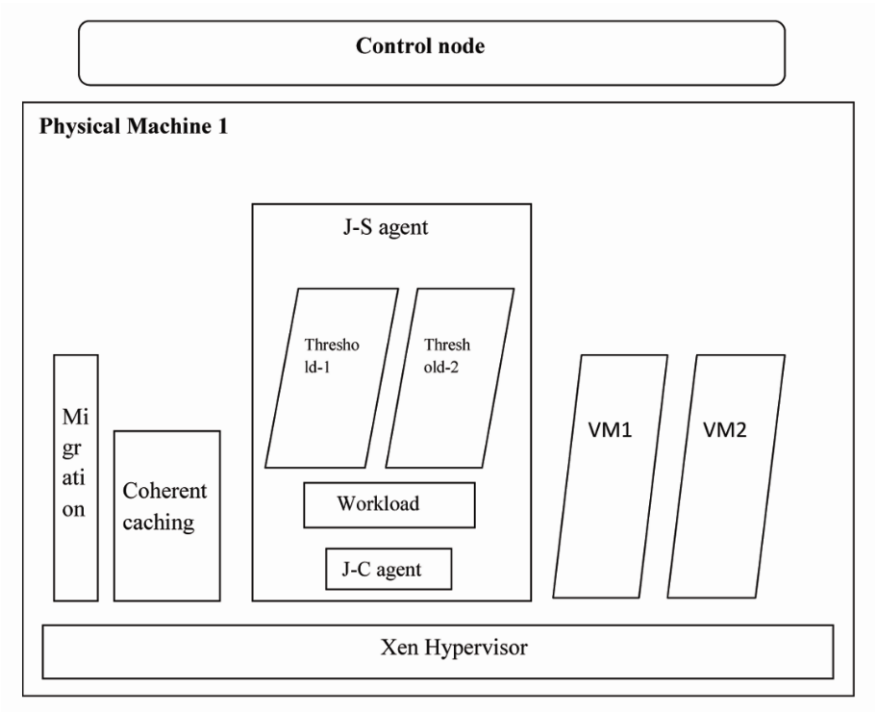

Fig 4: J-S agent

Figure 4 illustrates how the J-S agent, by determining the action of joint control and coherent caching, provides more room for the virtual machines and effectively addresses overload. The J-S agent saves data center resources for virtual machines by deciding between memory allocation and avoiding virtual machine oversubscription. In the 640 MB case, if physical machines are oversubscribed, more space is required to address sustained overloads, as seen in the 512 MB case.

Evaluation: The evaluation assessed:

a) The J-S agent's ability to make optimal decisions during heavy memory overload.

b) The J-S agent's effectiveness in lessening the burden of overloading, even when virtual machines are oversubscribed.

An experiment using SPEC Web 2009 with varying client loads was conducted to simulate real-world data center conditions and estimate the J-S agent's performance under different memory overload conditions.

Posted on 29 Apr 2025 — The copyright holder is the author/funder. All rights reserved. No reuse without permission. — https://doi.org/10.22541/au.174593665.52735213/v1 — This is a preprint and has not been peer-reviewed. Data may be preliminary.

Posted on 29 Apr 2025 — The copyright holder is the author/funder. All rights reserved. No reuse without permission. — https://doi.org/10.22541/au.174593665.52735213/v1 — This is a preprint and has not been peer-reviewed. Data may be preliminary.

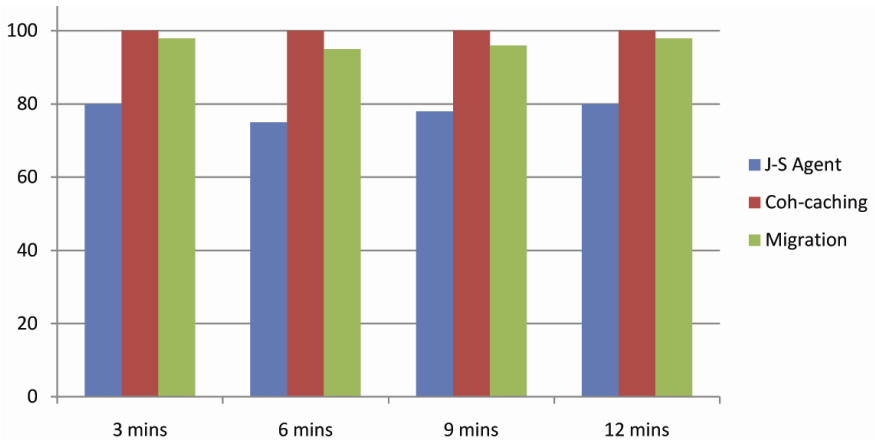

Fig 5: coherent caching migration J-S Agent and Coh-Catching and Migration

Figure 5 shows that coherent caching effectively competes with migration, aligning with the goal of minimizing virtual machine migration. The J-S agent, which controls the J-C (Joint Controller), effectively reduces migration, which in turn reduces server oversubscription.

Tab1: Details of Threshold, throughput and response times w.r.to Caching.

Threshold	Throughput(%)	Response Time(mins)
90	86.24	896
100	86.37	1150
110	84.29	1185
120	82.13	1114
150	80.10	1120
170	79.54	1256

Table 1 shows that performance degradation increases with higher thresholds, increasing the response time of coherent caching.

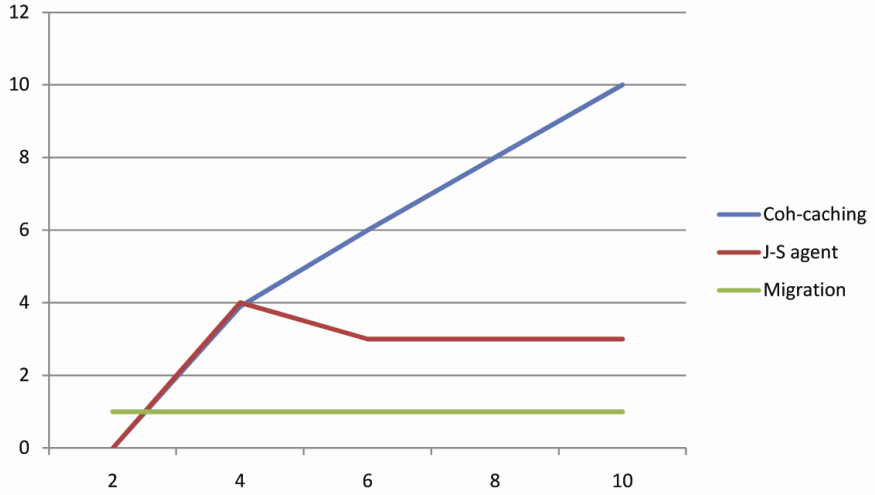

Fig 6: Functional Comparison of J-S Agent and Coh-Catching and Migration Line

Posted on 29 Apr 2025 — The copyright holder is the author/funder. All rights reserved. No reuse without permission. — https://doi.org/10.22541/au.174593665.52735213/v1 — This is a preprint and has not been peer-reviewed. Data may be preliminary.

Figure 5 and Figure 6 illustrate the amount of traffic sent and received for various client bursts. Coherent caching stores the swapped pages, and overload increases with the number of swapped pages. Although coherent caching increases network traffic by reading from page repositories, migration remains at a fixed rate. Memory allocation for virtual machines increases with increased migration.

Conclusion

In cloud computing environments, virtualization helps meet the large demands of clients. However, the increasing oversubscription of resources necessitates efficient management to address overloading and virtual machine migration without affecting cloud application performance. Two types of overloading were identified: transient and sustained. Transient overloads, lasting less than 2 minutes, can be addressed by virtualization. Sustained overloads create tradeoffs for data centers.

Coherent caching was introduced to address both sustained and transient overloads. The algorithm, using the J-S (Joint Solution) agent, addresses sustained overloads without violating SLAs and effectively manages oversubscription and migration. The J-S agent enables future cloud computing data centers to handle larger workloads.

References

1. Z. Zhang, X. Chen, and Y. Xiang, "Energy-Aware Virtual Machine Placement in Cloud Data Centers: A Survey," *IEEE Transactions on Parallel and Distributed Systems* , vol. 32, no. 5, pp. 1073-1092, May 2021.
2. S. Kumari and S. C. Sharma, "A Comprehensive Survey on Resource Management Techniques in Cloud Computing," *Journal of King Saud University - Computer and Information Sciences* , vol. 33, no. 8, pp. 915-931, Oct. 2021.
3. M. Aazam, I. Haque, and E. Y. K. Ng, "Adaptive Virtual Machine Consolidation for Energy Efficiency in Cloud Data Centers,"*Sustainable Computing: Informatics and Systems* , vol. 30, Jun. 2021, 100514.
4. H. Li, Z. Ouyang, and Y. Zhang, "Deep Reinforcement Learning for Dynamic Virtual Machine Resource Management in Cloud Computing,"*Future Generation Computer Systems* , vol. 115, pp. 314-324, Feb. 2021.
5. A. Verma, L. Pedrosa, M. Korolova, D. Nauck, Q. Le-Trung, I. Lalis, and M. Musolesi, "Learning to Schedule in Cloud Computing: The Gap, the Challenge, and the Promise," *IEEE Transactions on Cloud Computing* , vol. 9, no. 1, pp. 1-15, Jan.-Mar. 2021.
6. Y. Zhao, J. Wu, C. Huang, and T. Li, "A Survey of Virtual Machine Migration in Cloud Computing," *IEEE Access* , vol. 8, pp. 163438-163455, 2020.
7. B. Mao, F. Zhang, and S. Guo, "A Survey of Resource Provisioning Strategies in Cloud Computing," *Journal of Parallel and Distributed Computing* , vol. 140, pp. 1-18, Jun. 2020.
8. S. Bibi, A. K. Singh, and N. Kumar, "Energy-Efficient Resource Allocation Techniques in Cloud Computing: A Systematic Review,"*Journal of Network and Computer Applications* , vol. 164, Sep. 2020, 102677.
9. J. Zhou, Z. Xiao, Y. Zhang, and X. Zhou, "A Survey of Virtual Machine Placement Algorithms in Cloud Computing," *Journal of Cloud Computing: Advances and Systems* , vol. 8, no. 1, p. 19, Dec. 2019.
10. M. Shojafar, N. Javanmardi, N. Ameri, and A. Abraham, "Resource Provisioning Based on Fuzzy Logic in Cloud Computing: A Review and Future Directions," *Applied Soft Computing* , vol. 83, Nov. 2019, 105640.
11. K. Li, Y. Zhou, and F. Zhao, "Research on Virtual Machine Migration Technology in Cloud Computing," *IEEE Access* , vol. 7, pp. 117097-117111, 2019.
12. S. Sharma and R. K. Singh, "Virtual Machine Placement Techniques in Cloud Computing: A Comprehensive Review," *International Journal of Computer Applications* , vol. 179, no. 38, pp. 1-7, Sep. 2018.
13. V. V. Phoha, N. S. Dhillon, and S. Verma, "A Survey on Virtual Machine Scheduling Techniques for Cloud Computing," *International Journal of Parallel Programming* , vol. 46, no. 3, pp. 615-653, Jun.

2018.
14. M. Wazid, A. K. Das, R. P. Singh, and S. K. Verma, "A Survey on Energy-Aware Resource Management in Cloud Computing," *Journal of Grid Computing* , vol. 15, no. 1, pp. 133-163, Mar. 2017.
15. J. Gubbi, R. Buyya, S. Marusic, and M. Palaniswami, "Internet of Things (IoT): A Vision, Architectural Elements, and Future Directions," *Future Generation Computer Systems* , vol. 29, no. 7, pp. 1645-1660, Sep. 2013.
16. D. Nurmi, J. Lappalainen, and E. Helkala, "A Survey of Resource Management in Infrastructure-as-a-Service Cloud Computing,"*EURASIP Journal on Wireless Communications and Networking* , vol. 2012, no. 1, p. 246, Sep. 2012.
17. A. Beloglazov and R. Buyya, "Optimal Online Deterministic Algorithms and Adaptive Heuristics for Energy and Performance Efficient Dynamic Virtual Machine Consolidation in Cloud Data Centers,"*Concurrency and Computation: Practice and Experience* , vol. 24, no. 13, pp. 1397-1420, Sep. 2012.
18. R. N. Calheiros, S. K. Garg, Z. Zeng, R. Ranjan, C. A. F. De Rose, and R. Buyya, "Workload Management in Cloud Computing: Taxonomy and Future Directions," *Internet Computing* , vol. 15, no. 1, pp. 82-89, Jan.-Feb. 2011.
19. R. Buyya, A. K. Dastjerdi, R. N. Calheiros, and S. K. Garg, "Intercloud: Utility-Oriented Federation of Cloud Computing Environments for Scaling of Application Services," in*Proceedings of the 10th International Conference on Algorithms and Architectures for Parallel Processing* , Busan, South Korea, 2010, pp. 13-31.
20. L. M. Vaquero, L. Rodero-Merino, J. Caceres, and M. Lindner, "A Break in the Clouds: Towards a Cloud Definition," *ACM SIGCOMM Computer Communication Review* , vol. 39, no. 1, pp. 50-55, Jan. 2009.

Posted on 29 Apr 2025 — The copyright holder is the author/funder. All rights reserved. No reuse without permission. — https://doi.org/10.22541/au.174593665.52735213/v1 — This is a preprint and has not been peer-reviewed. Data may be preliminary.